

Macedonia Ride — Development Blueprint

Table of Contents

Macedonia Ride — Целосен Development Blueprint

Carpool / ride-sharing платформа за меѓуградско заедничко патување во Македонија. Верзија на документот: 1.0 · Датум: 30.06.2026 · Тип: техничка спецификација подготвена за развој.

Овој документ е целосна спецификација што може директно да се користи за програмирање. Содржи 27 секции: од концепт, преку база и API, до план за развој по фази.

1. Кратко објаснување на платформата

Macedonia Ride е mobile-first web платформа за меѓуградско carpool патување. За разлика од класичен Uber каде возачот вози специјално за патникот, овде возачите кои **веќе патуваат** од еден град во друг објавуваат слободни места во својот автомобил, а патниците резервираат место и патуваат заедно со нив, споделувајќи ги трошоците.

Главна вредност: безбедно, едноставно и евтино меѓуградско патување, со јасна рута, проверени корисници и live tracking на возилото.

Три улоги:

- **Патник (Passenger)** — пребарува патување, испраќа барање со pickup/dropoff локација, комуницира со возач, го следи возилото.
- **Возач (Driver)** — креира патување, дефинира рута и waypoints, управува со барања, започнува патување, споделува live локација.
- **Админ (Admin)** — управува со корисници, патувања, резервации, reports, градови/рути, претплати и промоции.

Клучни принципи:

1. **Request-to-Join, не инстант резервација** — возачот секогаш одобрува.
2. **Route-first ranking** — промоцијата никогаш не надминува релевантност на рутата.
3. **Privacy by default** — точната локација и телефон се откриваат само по одобрување.

4. **Cash-first MVP** — платформата прво поврзува луѓе; online плаќање доаѓа подоцна.
5. **Detour-aware matching** — системот пресметува колку pickup/dropoff отстапува од рутата.

Препорачан tech stack (детали во секција 23): Next.js + TypeScript + Tailwind + shadcn/ui на frontend; Supabase (PostgreSQL + Auth + Realtime + Storage) на backend; Mapbox или Google Maps за мапи, рути и геокодирање.

2. Sitemap

Public pages

/	Landing
/find-ride	Пребарување (public preview)
/offer-ride	Маркетинг за возачи
/how-it-works	Како функционира
/safety	Безбедност и доверба
/popular-routes	Популарни рути (SEO)
/login	
/register	
/forgot-password	
/terms	
/privacy	
/contact	

Passenger dashboard

/dashboard/passenger	Overview
/dashboard/passenger/find	Find Ride + филтри
/dashboard/passenger/trips	My Trips (tabs)
/dashboard/passenger/trips/[id]	Trip detail + live tracking
/dashboard/passenger/bookings	Резервации
/dashboard/passenger/messages	Chat
/dashboard/passenger/saved-locations	Зачувани локации
/dashboard/passenger/reviews	Оценки
/dashboard/passenger/profile	Профил и settings

Driver dashboard

/dashboard/driver	Overview
/dashboard/driver/create-trip	Креирај патување (wizard)
/dashboard/driver/trips	My Trips (tabs)
/dashboard/driver/trips/[id]	Trip detail
/dashboard/driver/requests	Барања од патници
/dashboard/driver/active-trip/[id]	Live trip контроли
/dashboard/driver/vehicles	Возила
/dashboard/driver/subscription	Претплата
/dashboard/driver/promotions	Промоции и статистика
/dashboard/driver/messages	Chat

/dashboard/driver/reviews	Оценки
/dashboard/driver/profile	Профил

Admin dashboard

/admin	Overview / KPIs
/admin/users	Сите корисници
/admin/drivers	Возачи + возила
/admin/passengers	Патници
/admin/trips	Патувања
/admin/bookings	Резервации
/admin/reports	Пријави
/admin/cities	Градови
/admin/routes	Рути + pickup/dropoff точки
/admin/subscriptions	Претплати
/admin/promotions	Промоции
/admin/settings	Системски подесувања

Shared / utility

/trip/share/[token]	Public read-only share view (Share Trip)
/verify-email	
/verify-phone	
/onboarding/role	Избор на улога по регистрација

3. User flows за патник

3.1 Регистрација → прв search

Register (email/phone) → verify → избор улога (патник) → пополни профил (град, слика) → Find Ride.

3.2 Find Ride → Request to Join

1. Внеси: почетен град, краен град, датум, време, бр. патници.
2. (опц.) Внеси pickup и dropoff локација (мапа / адреса / GPS / зачувана).
3. Системот враќа резултати: [Промовирани] и [Сите патувања], сортирани по route-match → promotion → време → рејтинг → близина (секција 18).
4. Отвори Trip Details: возач, авто, рута на мапа, правила, цена, слободни места.
(Регистарска табличка делумно скриена, телефон скриен.)
5. „Request to Join”: внеси seats, pickup, dropoff, багаж, порака.
6. Системот пресметува detour (мин) и distance-from-route (км) и означува дали е во рамки на max_detour_minutes на возачот.
7. Барање → status = pending. Се отвора chat (телефон сè уште скриен).

3.3 По одобрување

Driver approves → seats се намалуваат → telefon видлив → точна pickup видлива → notification + chat. Патникот гледа Trip во „Approved”.

3.4 Активно патување (live tracking)

Driver „Start Trip" → патникот гледа marker на мапа, ETA до pickup и до дестинација, статус, последно ажурирање. Може „Share Trip".

Статуси течат: driver_on_the_way → arrived_at_pickup → passenger_picked_up → in_progress → completed. По completed: tracking off → остави оценка.

3.5 Откажување / report

Cancel пред старт (cancelled_by_passenger) → seats се враќаат.

По патување: Report (no-show, опасно возење, итн.) + Review.

4. User flows за возач

4.1 Onboarding

Register → verify → улога (возач) → додади возило (My Vehicles) → (опц.) verification → Create Trip.

4.2 Create Trip (wizard, секција 6)

Step 1 Рута: почетен/краен град, точни старт/дестинација координати, route description, waypoints, попатни застанувања.

Step 2 Detour: detour_allowed, max_detour_minutes, can_pickup/can_dropoff по waypoint.

Step 3 Капацитет/цена: available_seats, price_per_seat.

Step 4 Правила: smoking, pets, luggage, AC, notes.

Step 5 Распоред: датум/време; еднократно или повторувачко (recurring rule).

Step 6 (опц.) Promote this Trip → boost/subscription.

→ Publish (status: published/scheduled).

4.3 Управување со барања

Requests: за секое барање — патник, рејтинг, бр. патници, pickup/dropoff на мапа, detour estimate, дали е во лимит, порака, багаж.

Акции: Accept | Reject | Message | Predloži друга pickup точка |

Predloži друго време | Cancel trip.

Accept → seats-- , patnik approved, telefon видлив за двете страни.

4.4 Active Trip контроли

Start Trip (бара GPS permission, почнува location sharing) →

Arrived at Pickup → Passenger Picked Up → Complete Trip.

Во секој момент: Cancel Trip. Complete/Cancel → tracking off.

4.5 По патување

Earnings се ажурираат (expected, cash MVP). Оцени ги патниците. Reviews.

5. User flows за admin

- 5.1 Login (admin role) → Overview KPIs.
 - 5.2 Модерација корисници: преглед профил → verify / block / activate.
 - 5.3 Возачи: преглед возила и verification → approve/reject verification.
 - 5.4 Патувања: list/filter → unpublish/delete suspicious trip.
 - 5.5 Резервации: преглед по статус → manual intervene при спор.
 - 5.6 Reports: inbox → отвори → истражи → resolve/escalate → (опц.) block.
 - 5.7 Cities/Routes: CRUD градови, popular routes, pickup/dropoff точки.
 - 5.8 Subscriptions/Promotions: планови CRUD, manual activate/deactivate, преглед статистика.
 - 5.9 Settings: fees, cancellation rules, verification rules, notification templates, promotion rules.
-

6. Рута на движење на возачот

Возачот мора јасно да ја дефинира рутата — не е доволно „Скопје → Охрид“. Дефиниција содржи:

- почетен/краен град + **точни** старт/дестинација координати и адреси;
- route_description (текстуален опис на главната рута);
- trip_waypoints — подредена листа места низ кои поминува, секое со can_pickup / can_dropoff;
- можни попатни застанувања;
- detour_allowed и max_detour_minutes;
- route_polyline (encoded polyline од Directions API, кеширан) за route-matching и приказ на мапа.

Пример (Скопје – Велес):

Старт: Скопје, Аеродром (41.9430, 21.5100)

Рута: Аеродром → автопат Е-75 → Катланово → Велес

Waypoints: Петровец (pickup✓ dropoff✓), Катланово (pickup✓ dropoff✓),
Башино Село (pickup× dropoff✓)

Правило: само од главната рута, без големо скршнување. max_detour = 10 мин.

Во Trip Details патникот ја гледа целата рута на мапа (polyline + waypoint markers).

7. Pickup и dropoff локација на патник

При барање патникот избира **pickup** и **dropoff** преку: мапа, пребарување адреса (geocoding), моментална GPS локација, зачувана локација, или предложена pickup точка од возачот.

Возачот ја гледа pickup/dropoff локацијата **пред** одобрување. Системот автоматски пресметува:

- дали pickup е блиску до рутата (distance-from-route во км);
- колку минути скршнување бара pickup (detour estimate);
- дали е во рамки на max_detour_minutes;
- истото за dropoff.

Пример 1 (во лимит): Рута Аеродром→Е75→Катланово→Велес, max 10 мин. Pickup: Кисела Вода → „8 минути скршнување — во дозволениот лимит."

Пример 2 (над лимит): Pickup: Сарај → „22 минути скршнување — над дозволениот лимит." Возачот сепак може рачно да прифати.

Детална математика во секции 16–17.

8. Privacy логика за локација

ПРАВИЛА:

- Возачот ја гледа pickup/dropoff само ОТКАКО патникот испратил барање за неговото патување (не пред тоа, не за други патувања).
- Возачот НЕ ја гледа live GPS на патникот цело време.
- Пред одобрување: прикажи ПРИБЛИЖНА зона (snapped кон најблиска јавна точка / заокружени координати ~300–500 м).
- По одобрување: прикажи ТОЧНА pickup локација.
- За домашна адреса: патникот добива опција „nearby public pickup point".
- Патникот може РАЧНО да сподели live локација, само ако сака.
- По completed/cancelled: локацијата НЕ се споделува активно.
- Телефонски број: скриен до approved booking.

Имплементација: чувствителните полиња се сервираат преку API слој кој применува правила според booking.status и улогата на барателот; на ниво на база — RLS политики (секција 13).

9. Барање за резервација (Request to Join)

Нема автоматска резервација. Патникот испраќа барање со: seats_requested, pickup (адреса+коорд+note), dropoff (адреса+коорд+note), message, багаж инфо, посебно барање.

Статуси на booking:

pending → approved | rejected

approved → cancelled_by_passenger | cancelled_by_driver | completed | no_show

Кога возачот одобри, `trips.available_seats` се намалува за `seats_requested` (атомски, во транзакција, со проверка дека има доволно места).

10. Chat систем

Внатрешен chat возач ↔ патник, врзан за booking. Достапен: при испратено барање, додека е `pending`, по `approved`, за време на активно патување и кратко по `completed` (пр. 48 ч).

Типови пораки (`message_type`): `text`, `pickup_clarification`, `system`, `location_suggestion`, `trip_update` (automated). Телефон не се прикажува веднаш — само по `approved`.

Realtime преку Supabase Realtime (Postgres changes на messages) или WebSocket канал по `booking_id`.

11. Live tracking на возилото

Кога возачот кликне **Start Trip**, неговата GPS локација се споделува во реално време со патници со **approved** booking.

Патникот гледа: позиција на возилото, движење на мапа, дали се движи кон pickup, ETA до pickup, ETA до дестинација, последно ажурирање, статус, дали стои или се движи, копче „Share Trip“.

ТЕХНИЧКА ЛОГИКА:

1. Driver → Start Trip.
2. status → driver_on_the_way (/ in progress).
3. Web app бара GPS permission (`navigator.geolocation.watchPosition`).
4. Локација се испраќа на секои ~3-5 сек (`throttle/debounce`).
5. Последна локација → trip_locations + Realtime broadcast (low-latency).
6. Само approved passengers смеат да читаат (RLS / channel auth).
7. Патник гледа marker што се движи (Mapbox GL).
8. Систем пресметува ETA (Directions API или хаверсин + просечна брзина).
9. Complete/Cancel → tracking off, престанува broadcast.

Забелешка: за web MVP tracking работи додека возачот ја има отворено страницата (Wake Lock API помага). Вистинско background tracking бара native Android/iOS app (future).

Архитектурата е детална во секција 14.

12. Статуси на патување

draft → published → scheduled → driver_on_the_way → arrived_at_pickup → passenger_picked_up → in_progress → completed
(во секој активен момент: → cancelled)

Возачки копчиња: Publish Trip · Start Trip · Arrived at Pickup · Passenger Picked Up · Complete Trip · Cancel Trip. Патникот добива notification на секоја промена.

13. Admin панел структура

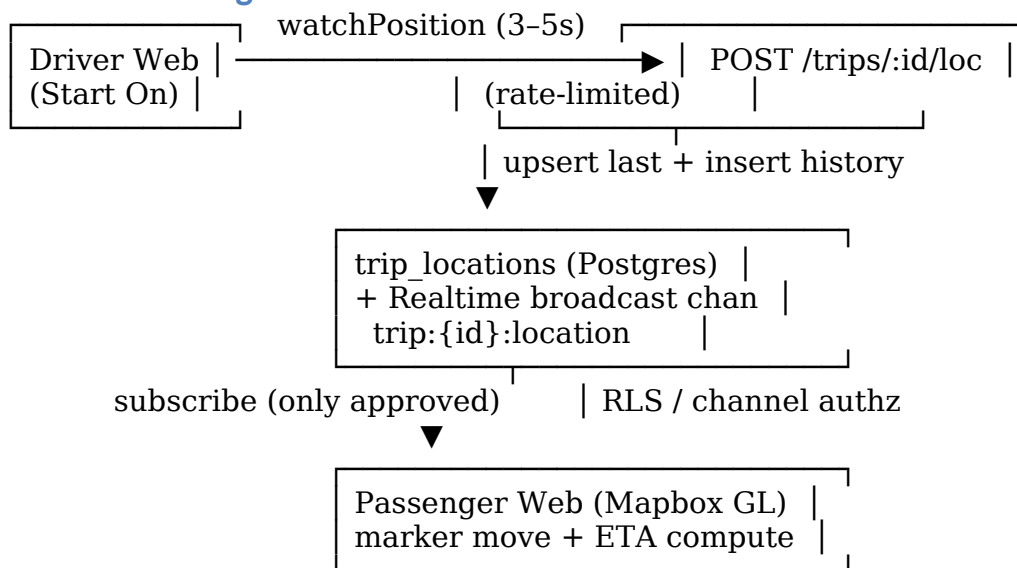
Overview KPIs: вкупно корисници / возачи / патници, активни патувања, завршени патувања, pending reports, најпопуларни рути, активни promoted rides, активни subscriptions.

Модули: Users · Drivers · Passengers · Trips · Bookings · Reports · Cities · Routes · Subscriptions · Promotions · Settings (детален опис на секции и акции е даден во оригиналната спецификација; мапиран е во sitemap-от и API-то подолу).

Reports причини: возач/патник не се појавил, опасно возење, лажно патување, навредливо однесување, измама, погрешна рута, проблем со плаќање.

Settings: fees, cancellation rules, verification rules, notification templates, privacy settings, promotion rules — чувани во admin_settings (key/value).

14. Live tracking architecture



Решенија:

- **Транспорт:** Supabase Realtime Broadcast (ниска латентност, не оптоварува insert на секоја точка) + повремен trip_locations insert за историја/recovery.
 - **Authz:** Realtime канал trip:{tripId} со проверка дека корисникот има approved booking или е возачот.
 - **ETA:** примарно Directions API (трае, кешира), fallback Haversine растојание ÷ просечна брзина по тип пат.
 - **Battery/permission:** Wake Lock API, throttling, „resume tracking“ prompt ако табот се успие.
 - **Cleanup:** при completed/cancelled — отповикај канал, стопирај watchPosition, маркирај крај.
-

15. Детална database schema (целосен SQL)

PostgreSQL (Supabase). Користени се ENUM типови, uuid PK, timestamps и индекси. Геопросторот може да се чува како double precision lat/lng (MVP) или PostGIS geography(Point) (препорачано за route-matching — види секција 17). Подолу е MVP верзија со lat/lng + опционални PostGIS колони.

```
-- ===== EXTENSIONS =====
create extension if not exists "uuid-ossf";
create extension if not exists postgis; -- за route-matching (препорачано)

-- ===== ENUMS =====
create type user_role as enum ('passenger','driver','admin');
create type user_status as enum ('active','blocked','pending','deleted');
create type verification_st as enum ('unverified','pending','verified','rejected');
create type trip_status as enum ('draft','published','scheduled',
                                'driver_on_the_way','arrived_at_pickup',
                                'passenger_picked_up','in_progress',
                                'completed','cancelled');
create type booking_status as enum ('pending','approved','rejected',
                                    'cancelled_by_passenger','cancelled_by_driver',
                                    'completed','no_show');
create type message_type as enum ('text','pickup_clarification','system',
                                  'location_suggestion','trip_update');
create type review_type as enum ('driver_to_passenger','passenger_to_driver');
create type report_status as enum ('open','investigating','resolved','dismissed');
create type promotion_type as enum
('subscription','boost_24h','boost_3d','featured');
create type promotion_status as enum ('active','expired','cancelled','scheduled');
create type sub_status as enum ('active','expired','cancelled','past_due');
create type pay_status as enum ('unpaid','paid','refunded','pending');
create type recurrence_type as enum
('none','weekdays','weekends','weekly','custom');

-- ===== USERS =====
```

```

create table users (
  id          uuid primary key default uuid_generate_v4(),
  first_name   text not null,
  last_name    text not null,
  email        text unique,
  phone        text unique,
  password_hash text,
  profile_image text,
  city_id      uuid references cities(id),
  role        user_role not null default 'passenger',
  is_driver    boolean not null default false,
  is_passenger boolean not null default true,
  email_verified boolean not null default false,
  phone_verified boolean not null default false,
  status       user_status not null default 'active',
  created_at   timestampz not null default now(),
  updated_at   timestampz not null default now()
);
create index idx_users_role on users(role);
create index idx_users_status on users(status);

-- ===== CITIES =====
create table cities (
  id          uuid primary key default uuid_generate_v4(),
  name         text not null unique,
  latitude     double precision,
  longitude     double precision,
  geom         geography(Point,4326),
  is_active    boolean not null default true
);

-- ===== PASSENGER SAVED LOCATIONS =====
create table passenger_saved_locations (
  id          uuid primary key default uuid_generate_v4(),
  user_id      uuid not null references users(id) on delete cascade,
  label       text not null,
  address      text,
  latitude     double precision not null,
  longitude     double precision not null,
  geom         geography(Point,4326),
  note         text,
  is_default    boolean not null default false,
  created_at   timestampz not null default now()
);
create index idx_saved_loc_user on passenger_saved_locations(user_id);

-- ===== DRIVER PROFILES =====
create table driver_profiles (
  id          uuid primary key default uuid_generate_v4(),

```

```

user_id      uuid not null unique references users(id) on delete cascade,
bio          text,
rating_average numeric(3,2) not null default 0,
total_trips  integer not null default 0,
verification_status verification_st not null default 'unverified',
created_at   timestampz not null default now()
);

```

```

-- ===== VEHICLES =====

```

```

create table vehicles (
  id          uuid primary key default uuid_generate_v4(),
  driver_id    uuid not null references users(id) on delete cascade,
  make         text not null,
  model        text not null,
  year         integer,
  color        text,
  license_plate text not null,
  seats        integer not null check (seats between 1 and 8),
  has_ac        boolean not null default false,
  luggage_space text,           -- none / small / medium / large
  vehicle_image text,
  is_active     boolean not null default true,
  created_at    timestampz not null default now()
);
create index idx_vehicles_driver on vehicles(driver_id);

```

```

-- ===== TRIPS =====

```

```

create table trips (
  id          uuid primary key default uuid_generate_v4(),
  driver_id    uuid not null references users(id) on delete cascade,
  vehicle_id    uuid references vehicles(id),
  start_city_id  uuid not null references cities(id),
  end_city_id    uuid not null references cities(id),
  start_address  text,
  start_latitude double precision not null,
  start_longitude double precision not null,
  destination_address text,
  destination_latitude double precision not null,
  destination_longitude double precision not null,
  route_polyline text,           -- encoded polyline (кеширан)
  route_geom     geography(LineString,4326),
  departure_date  date not null,
  departure_time  time not null,
  available_seats integer not null check (available_seats >= 0),
  total_seats     integer not null,
  price_per_seat  numeric(10,2) not null,
  route_description text,
  detour_allowed  boolean not null default true,
  max_detour_minutes integer not null default 10,
  smoking_allowed boolean not null default false,

```

```

pets_allowed      boolean not null default false,
luggage_allowed   boolean not null default true,
notes             text,
status            trip_status not null default 'draft',
is_recurring       boolean not null default false,
recurrence         recurrence_type not null default 'none',
recurrence_config  jsonb,           -- {days:[1,3,5], until:'2026-12-31'}
parent_trip_id     uuid references trips(id), -- 3a recurring instances
is_promoted        boolean not null default false,
promotion_priority integer not null default 0,
promoted_until     timestampz,
created_at         timestampz not null default now(),
updated_at         timestampz not null default now()
);
create index idx_trips_route on trips(start_city_id, end_city_id, departure_date);
create index idx_trips_status on trips(status);
create index idx_trips_promo on trips(is_promoted, promotion_priority desc);
create index idx_trips_driver on trips(driver_id);
create index idx_trips_routegeom on trips using gist(route_geom);

-- ===== TRIP WAYPOINTS =====
create table trip_waypoints (
  id          uuid primary key default uuid_generate_v4(),
  trip_id     uuid not null references trips(id) on delete cascade,
  city_id     uuid references cities(id),
  address     text,
  latitude    double precision not null,
  longitude   double precision not null,
  geom        geography(Point,4326),
  order_index integer not null,
  can_pickup  boolean not null default true,
  can_dropoff boolean not null default true
);
create index idx_waypoints_trip on trip_waypoints(trip_id, order_index);

-- ===== BOOKINGS =====
create table bookings (
  id          uuid primary key default uuid_generate_v4(),
  trip_id     uuid not null references trips(id) on delete cascade,
  passenger_id uuid not null references users(id) on delete cascade,
  seats_requested integer not null default 1 check (seats_requested > 0),
  pickup_address text,
  pickup_latitude double precision not null,
  pickup_longitude double precision not null,
  pickup_geom     geography(Point,4326),
  pickup_note     text,
  dropoff_address text,
  dropoff_latitude double precision not null,
  dropoff_longitude double precision not null,
  dropoff_geom     geography(Point,4326),

```

```

dropoff_note          text,
detour_minutes_estimate  numeric(6,2),
distance_from_route_km  numeric(7,3),
is_within_driver_detour_limit boolean,
message               text,
luggage_info          text,
status                booking_status not null default 'pending',
created_at            timestampz not null default now(),
updated_at            timestampz not null default now(),
unique (trip_id, passenger_id)      -- едно барање по патување по патник
);
create index idx_bookings_trip on bookings(trip_id);
create index idx_bookings_pass on bookings(passenger_id);
create index idx_bookings_status on bookings(status);

-- ===== MESSAGES =====
create table messages (
  id          uuid primary key default uuid_generate_v4(),
  trip_id     uuid references trips(id) on delete cascade,
  booking_id  uuid references bookings(id) on delete cascade,
  sender_id   uuid not null references users(id),
  receiver_id uuid not null references users(id),
  message     text not null,
  message_type message_type not null default 'text',
  created_at  timestampz not null default now(),
  read_at     timestampz
);
create index idx_messages_booking on messages(booking_id, created_at);

-- ===== TRIP LOCATIONS (live tracking) =====
create table trip_locations (
  id          bigserial primary key,
  trip_id     uuid not null references trips(id) on delete cascade,
  driver_id   uuid not null references users(id),
  latitude    double precision not null,
  longitude   double precision not null,
  speed       double precision,
  heading     double precision,
  recorded_at timestampz not null default now()
);
create index idx_triploc_trip on trip_locations(trip_id, recorded_at desc);

-- ===== REVIEWS =====
create table reviews (
  id          uuid primary key default uuid_generate_v4(),
  trip_id     uuid not null references trips(id) on delete cascade,
  booking_id  uuid references bookings(id) on delete cascade,
  reviewer_id uuid not null references users(id),
  reviewed_user_id uuid not null references users(id),

```

```

rating      integer not null check (rating between 1 and 5),
punctuality integer check (punctuality between 1 and 5),
driving_safety integer check (driving_safety between 1 and 5),
communication integer check (communication between 1 and 5),
cleanliness integer check (cleanliness between 1 and 5),
behavior     integer check (behavior between 1 and 5),
comment      text,
review_type  review_type not null,
created_at   timestampz not null default now(),
unique (booking_id, reviewer_id, review_type)
);

-- ===== REPORTS =====
create table reports (
  id          uuid primary key default uuid_generate_v4(),
  reporter_id  uuid not null references users(id),
  reported_user_id uuid references users(id),
  trip_id      uuid references trips(id),
  booking_id   uuid references bookings(id),
  reason       text not null,      -- enum-like: no_show / dangerous / fake ...
  description   text,
  status        report_status not null default 'open',
  created_at    timestampz not null default now(),
  resolved_at   timestampz
);
create index idx_reports_status on reports(status);

-- ===== NOTIFICATIONS =====
create table notifications (
  id          uuid primary key default uuid_generate_v4(),
  user_id      uuid not null references users(id) on delete cascade,
  title        text not null,
  message      text,
  type         text not null,      -- booking_request / approved / trip_started ...
  data         jsonb,             -- {trip_id, booking_id, ...} 3a deep-link
  is_read      boolean not null default false,
  created_at   timestampz not null default now()
);
create index idx_notif_user on notifications(user_id, is_read, created_at desc);

-- ===== SUBSCRIPTION PLANS =====
create table subscription_plans (
  id          uuid primary key default uuid_generate_v4(),
  name         text not null,
  description   text,
  price         numeric(10,2) not null,
  currency      text not null default 'MKD',
  duration_days integer not null,
  promoted_rides_limit integer not null default 0,

```

```

priority_level    integer not null default 0,
is_active         boolean not null default true,
created_at        timestampz not null default now()
);

-- ===== DRIVER SUBSCRIPTIONS =====
create table driver_subscriptions (
  id              uuid primary key default uuid_generate_v4(),
  driver_id       uuid not null references users(id) on delete cascade,
  plan_id         uuid not null references subscription_plans(id),
  status          sub_status not null default 'active',
  start_date      timestampz not null default now(),
  end_date        timestampz not null,
  promoted_rides_used integer not null default 0,
  promoted_rides_limit integer not null default 0,
  payment_status  pay_status not null default 'unpaid',
  created_at      timestampz not null default now(),
  updated_at      timestampz not null default now()
);
create index idx_subs_driver on driver_subscriptions(driver_id, status);

-- ===== TRIP PROMOTIONS =====
create table trip_promotions (
  id              uuid primary key default uuid_generate_v4(),
  trip_id         uuid not null references trips(id) on delete cascade,
  driver_id       uuid not null references users(id),
  subscription_id  uuid references driver_subscriptions(id),
  promotion_type  promotion_type not null,
  priority_level  integer not null default 1,
  status          promotion_status not null default 'active',
  start_date      timestampz not null default now(),
  end_date        timestampz not null,
  impressions_count integer not null default 0,
  clicks_count    integer not null default 0,
  booking_requests_count integer not null default 0,
  created_at      timestampz not null default now(),
  updated_at      timestampz not null default now()
);
create index idx_promo_active on trip_promotions(status, end_date);

-- ===== ADMIN SETTINGS =====
create table admin_settings (
  id              uuid primary key default uuid_generate_v4(),
  setting_key     text not null unique,
  setting_value   jsonb not null,
  updated_at      timestampz not null default now()
);

```

Корисни тригери (skeleton):


```

-- auto-update updated_at
create or replace function set_updated_at() returns trigger as $$
begin new.updated_at = now(); return new; end; $$ language plpgsql;

create trigger trg_trips_updated before update on trips
  for each row execute function set_updated_at();
create trigger trg_bookings_updated before update on bookings
  for each row execute function set_updated_at();

-- намали seats при approve, врати при cancel (поедноставено)
create or replace function on_booking_status_change() returns trigger as $$
begin
  if new.status = 'approved' and old.status <> 'approved' then
    update trips set available_seats = available_seats - new.seats_requested
      where id = new.trip_id and available_seats >= new.seats_requested;
    if not found then raise exception 'Not enough seats'; end if;
  elsif old.status = 'approved'
    and new.status in ('cancelled_by_passenger', 'cancelled_by_driver', 'no_show')
  then
    update trips set available_seats = available_seats + new.seats_requested
      where id = new.trip_id;
    end if;
  return new;
end; $$ language plpgsql;

create trigger trg_booking_seats after update on bookings
  for each row execute function on_booking_status_change();

```

16. Relations между табелите

```

users 1—* vehicles          (driver_id)
users 1—1 driver_profiles   (user_id)
users 1—* passenger_saved_locations (user_id)
users 1—* trips             (driver_id)
users 1—* bookings          (passenger_id)
users *—1 cities            (city_id)

```

```

cities 1—* trips (start_city_id, end_city_id)
cities 1—* trip_waypoints

```

```

trips 1—* trip_waypoints
trips 1—* bookings
trips 1—* trip_locations
trips 1—* messages
trips 1—* trip_promotions
trips *—1 vehicles

```

```

bookings 1—* messages

```


bookings 1—* reviews

bookings *—1 trips

bookings *—1 users (passenger)

reviews *—1 users (reviewer_id, reviewed_user_id)

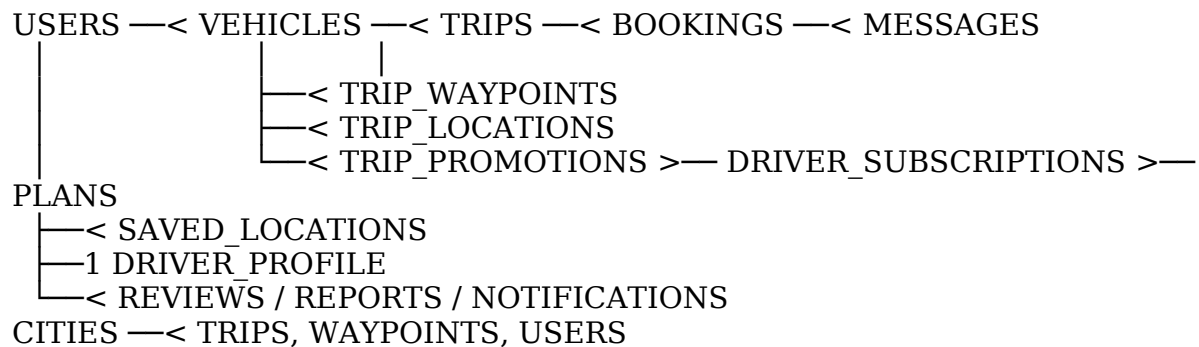
reports *—1 users (reporter_id, reported_user_id)

subscription_plans 1—* driver_subscriptions

driver_subscriptions 1—* trip_promotions

users (driver) 1—* driver_subscriptions

ER скица (текст):



17. Pages и components

Reusable UI components (shadcn/ui base):

RideCard — резултат во search (возач, рута, цена, places, badge)
RideCardPromoted — варијанта со „Промовирано" badge + highlight
TripStatusBadge — обоен статус chip
RouteMap — Mapbox GL: polyline + waypoint markers
LocationPicker — мапа + search + GPS + saved locations
DetourIndicator — „8 мин скршнување · во лимит" (зелено/црвено)
SeatSelector — stepper за seats
BookingRequestForm — pickup/dropoff/seats/message/багаж
RequestCard — за возач: патник + pickup/dropoff на мини-мапа + акции
ChatThread / ChatInput
LiveTrackingMap — moving marker + ETA panel
ReviewForm / ReviewStars
NotificationBell / NotificationList
FilterPanel — сите филтри за search
RecurrencePicker — еднократно / weekdays / weekends / custom
VehicleForm / VehicleCard
SubscriptionPlanCard / PromoteTripModal
ShareTripSheet — генерира share линк
ProfileForm / VerificationBadge
DataTable — admin листи (sort/filter/paginate)
KpiCard / StatChart — admin overview

Page – components мапа (примери):

/dashboard/passenger/find → FilterPanel, RideCard(+Promoted), RouteMap, Pagination
/dashboard/passenger/trips/[id] → RouteMap/LiveTrackingMap, TripStatusBadge, ChatThread, ReviewForm, ShareTripSheet
/dashboard/driver/create-trip → multi-step wizard: LocationPicker, RouteMap, RecurrencePicker, DetourSettings, rules
/dashboard/driver/requests → RequestCard list, DetourIndicator, accept/reject
/dashboard/driver/active-trip/[id] → LiveTrackingMap (driver), status контроли
/admin/* → DataTable, KpiCard, StatChart, detail drawers

18. Dashboard структура

Passenger Overview: следно патување, pending барања, approved патувања, quick search, recent trips.

Driver Overview: денешни патувања, upcoming, pending requests, approved passengers, expected earnings, rating, subscription status.

Admin Overview: KPI картички

(корисници/возачи/патници/активни/завршени/pending reports), најпопуларни рути, активни promoted rides, активни subscriptions, графикони (trips/day, bookings/day).

Layout: лев sidebar со навигација по улога, горен бар со NotificationBell + профил, content површина mobile-first (sidebar – bottom nav на мобилен).

19. API endpoints

REST-style (Next.js Route Handlers /app/api/... или Supabase Edge Functions). Сите враќаат JSON; заштитени со auth + RLS.

AUTH

POST	/api/auth/register	{first,last,email phone,password,role}
POST	/api/auth/login	{identifier,password}
POST	/api/auth/logout	
POST	/api/auth/forgot-password	{email}
POST	/api/auth/reset-password	{token,password}
POST	/api/auth/verify-email	{token}
POST	/api/auth/verify-phone	{code}
POST	/api/auth/role	{is_driver,is_passenger}

USERS / PROFILE

GET	/api/me	
PATCH	/api/me	профил, privacy, slika
GET	/api/users/:id	public profile (без чувствителни)

GET /api/me/saved-locations
POST /api/me/saved-locations
PATCH /api/me/saved-locations/:id
DELETE /api/me/saved-locations/:id

VEHICLES

GET /api/vehicles
POST /api/vehicles
PATCH /api/vehicles/:id
DELETE /api/vehicles/:id

TRIPS (driver)

POST /api/trips create (draft)
PATCH /api/trips/:id update
POST /api/trips/:id/publish
POST /api/trips/:id/cancel
GET /api/trips/mine driver's trips (tabs)
POST /api/trips/:id/start → driver_on_the_way (tracking on)
POST /api/trips/:id/arrived
POST /api/trips/:id/picked-up
POST /api/trips/:id/complete

SEARCH (passenger)

GET /api/search/rides?from=&to=&date=&time=&seats=
&pickupLat=&pickupLng=&dropoffLat=&dropoffLng=
&maxPrice=&minRating=&smoking=&pets=&ac=&luggage=&sort=
→ {promoted:[...], all:[...]} (ranking секција 21)
GET /api/trips/:id trip details (privacy-aware)

BOOKINGS

POST /api/trips/:id/bookings request to join (pickup/dropoff/seats/msg)
GET /api/bookings/mine passenger
GET /api/trips/:id/bookings driver: барања за патувањето
POST /api/bookings/:id/approve
POST /api/bookings/:id/reject
POST /api/bookings/:id/cancel (passenger/driver)
POST /api/bookings/:id/no-show
POST /api/bookings/:id/suggest {pickup?|time?} предлог од возач

GEO / MATCHING

POST /api/geo/detour {tripId,pickup,dropoff}
→ {detourMinutes, distanceFromRouteKm, withinLimit}
GET /api/geo/geocode?q= адреса → координати
GET /api/geo/route?from=&to=&via= Directions → polyline + waypoints

LIVE TRACKING

POST /api/trips/:id/location {lat,lng,speed,heading} (driver, throttled)
GET /api/trips/:id/location last known (approved passengers)
+ Realtime channel: trip:{id}:location

CHAT

GET /api/bookings/:id/messages
POST /api/bookings/:id/messages {message,type}
POST /api/messages/:id/read

REVIEWS

POST /api/bookings/:id/reviews {rating, sub-ratings, comment, type}
GET /api/users/:id/reviews

REPORTS / NOTIFICATIONS

POST /api/reports {reportedUserId?,tripId?,reason,description}
GET /api/notifications
POST /api/notifications/:id/read

MONETIZATION

GET /api/plans
POST /api/subscriptions subscribe (MVP: manual/cash → admin activate)
GET /api/me/subscription
POST /api/trips/:id/promote {type:boost_24h|boost_3d|featured}
GET /api/trips/:id/promotion/stats

ADMIN (role=admin)

GET /api/admin/overview
GET /api/admin/users PATCH /api/admin/users/:id (block/activate/verify)
GET /api/admin/drivers POST /api/admin/drivers/:id/verify
GET /api/admin/trips POST /api/admin/trips/:id/unpublish
GET /api/admin/bookings
GET /api/admin/reports POST /api/admin/reports/:id/resolve
CRUD /api/admin/cities /api/admin/routes
CRUD /api/admin/plans POST /api/admin/subscriptions/:id/activate
GET /api/admin/promotions
GET /api/admin/settings PATCH /api/admin/settings

20. Authentication, authorization, security & privacy

Auth (Supabase Auth): email+password и phone+OTP. JWT сесии. Email verification (magic link/token), phone verification (SMS OTP, провајдер пр. Twilio/локален SMS gateway). Forgot/reset password преку токен. Корисник избира улога; is_driver/is_passenger дозволуваат двојна улога.

Authorization:

- Middleware проверува сесија на сите /dashboard/* и /admin/*.
- Role gating: admin рути бараат role='admin'.
- Resource ownership: возач може да менува само свои trips/vehicles; патник само свои bookings/saved-locations.
- Row Level Security (RLS) во Postgres е втор слој (defense in depth).

RLS примери (Supabase):

```
alter table trips enable row level security;
```

```
alter table bookings enable row level security;
```

```
alter table trip_locations enable row level security;
```

```
alter table messages enable row level security;
```

```
-- Trips: сите гледаат published; возачот менаџира свои
```

```
create policy trips_select_public on trips for select
```

```
  using (status in ('published','scheduled') or driver_id = auth.uid());
```

```
create policy trips_modify_owner on trips for all
```

```
  using (driver_id = auth.uid()) with check (driver_id = auth.uid());
```

```
-- Bookings: само патникот сопственик или возачот на патувањето
```

```
create policy bookings_access on bookings for select using (
```

```
  passenger_id = auth.uid()
```

```
  or exists (select 1 from trips t where t.id = bookings.trip_id  
    and t.driver_id = auth.uid())
```

```
);
```

```
create policy bookings_passenger_insert on bookings for insert
```

```
  with check (passenger_id = auth.uid());
```

```
-- Trip locations: возачот пишува; читаат само approved passengers + возачот
```

```
create policy triploc_driver_write on trip_locations for insert
```

```
  with check (driver_id = auth.uid());
```

```
create policy triploc_read on trip_locations for select using (
```

```
  driver_id = auth.uid()
```

```
  or exists (select 1 from bookings b  
    where b.trip_id = trip_locations.trip_id  
    and b.passenger_id = auth.uid()  
    and b.status = 'approved')
```

```
);
```

```
-- Messages: само sender/receiver
```

```
create policy messages_access on messages for select
```

```
  using (sender_id = auth.uid() or receiver_id = auth.uid());
```

Privacy слој (секција 8) во API:

- Trip details: пред booking → license_plate маскирана (СК-***-AA), телефон скриен. По approved → целосни.
- Pickur точна локација: само возачот по испратено барање; патникот ја гледа својата. Пред approve може приближна зона.
- Phone reveal само при booking.status='approved'.

Security мерки: input validation (Zod), rate limiting (search, location POST, auth), CSRF за cookie сесии, HTTPS-only, hashing на password (ако custom), audit log за admin акции, content moderation hooks, soft-delete за корисници, GDPR-style data export/delete (future). Verified driver badge, report/block, share-trip — за доверба.

21. Route matching, detour calculation & search ranking

21.1 Route matching (дали патувањето одговара на патникот):

1. Hard filter: start_city/end_city директно ИЛИ патникот сака сегмент што е покриен од рутата (преку waypoints / polyline).
2. Geo match: pickup_point близу до route_polyline ($\leq$ threshold км) и dropoff_point близу, и dropoff е „после“ pickup по насока на рутата.
3. Date/time прозорец и available_seats $\geq$ seats_requested.
4. Правила: ако патникот филтрира non-smoking/pets/ac — applied.

21.2 Detour calculation:

distance_from_route_km = најкусо растојание од pickup точка до polyline (PostGIS ST_Distance(route_geom, pickup_geom)).
detour_minutes = T(route со pickup) – T(route без pickup)
- MVP апроксимација: $2 \times \text{distance_from_route_km} / \text{avg_speed_kmh} \times 60$
- Точно: Directions API со waypoint (кешира; повикувај штедливо).
within_limit = detour_minutes $\leq$ trips.max_detour_minutes

PostGIS пример:

select

```
st_distance(t.route_geom, b_pickup) / 1000.0 as dist_km
from trips t
where t.id = $1;
-- b_pickup := ST_SetSRID(ST_MakePoint(lng,lat),4326)::geography
```

Резултатот се чува во bookings.detour_minutes_estimate, distance_from_route_km, is_within_driver_detour_limit. Возачот секогаш може рачно да override-не.

21.3 Search ranking (sorting):

ПОРЕДОК:

1. Promoted rides со ТОЧЕН route match
2. Promoted rides преку попатна рута (segment match)
3. Обични: по време на тргнување (близина до бараното време)
4. Обични: по цена на возач
5. Обични: по близина на pickup/dropoff до рутата (помал detour прв)
6. Обични: по цена (само ако патникот избрал sort=price)

ПРАВИЛО: route-match е ПРЕДУСЛОВ за promotion boost. Нерелевантно промовирано патување (Скопје→Охрид за барање Скопје→Велес) НЕ оди најгоре освен ако реално поминува низ Велес и дозволува попатни патници.

Scoring (псевдо):

```
score = w_promo*promoQualified*priority_level
      + w_time*timeProximity
      + w_rating*driverRating
      + w_detour*(1/(1+detourMin))
      - w_price*priceNormalized(ако sort=price)
```

Резултат во две секции: { „Промовирани“, „Сите патувања“ } со badge.

22. Subscription & monetization логика

Типови возачи: Free · Basic Promotion · Premium · Pay-per-Boost.

Free — бесплатно објавува, стандардно сортирање, basic stats.

Basic Promotion — месечна претплата, X promoted rides/месец, „Promoted“ badge, повисоко рангирање, basic view/request stats.

Premium — повеќе promoted rides, Premium badge, advanced stats, повеќе видливост на популарни рути, featured profile.

Pay-per-Boost — еднократно: boost 24h / 3 дена / топ до денот на патување.

Promotion lifecycle: автоматски expired кога: патувањето започне, се откаже, се пополни (available_seats=0), или истече периодот. Cron/Edge Function проверува trip_promotions секои N мин.

Други модели (подготвени, MVP-flagged): platform fee по booking (20–30 ден/процент), service fee за патник, premium driver profile, featured routes (Скопје–Велес...), corporate packages (company dashboard, private routes, invoices), student packages, sponsored banners (пумпи/сервиси/осигурување), partnerships, airport rides категорија, monthly pass за патници, route alerts premium.

MVP монетизација: Free plan + Pay-per-Boost + Promoted badge + basic promoted sorting; subscription планови постојат во база, но online плаќање е future (manual/cash активација од admin за почеток).

23. Notifications логика

Настани → notification: ново барање, прифатено/одбиено барање, откажано патување, промена на време, trip started, arrived at pickup, passenger picked up, trip completed, „остави оцена“, chat порака, promoted ride истекува, subscription истекува.

Канали: in-app (табела notifications + Realtime bell) — MVP; email (transactional, пр. Resend/SMTP) — MVP за клучни; push (web push/FCM) — future; SMS — future, само критични.

Имплементација: на server акција (approve/start/...) се вметнува ред во notifications + (опц.) се праќа email преку queue/Edge Function. Frontend слуша Realtime на notifications по user_id.

24. MVP scope

PASSENGER MVP: register/login, профил, search, trip details, request-to-join (pickup/dropoff), chat, my trips, review.

DRIVER MVP: register/login, профил, vehicle, create trip (пута+waypoints +detour settings), requests, гледа pickup/dropoff, accept/reject, start/complete trip, review.

ADMIN MVP: users, drivers, trips, bookings, reports, cities, basic promoted rides management.

LIVE TRACKING MVP: tracking додека возачот има отворено страница, marker на мапа, само approved passengers, auto-off по completed/cancelled.

MONETIZATION MVP: Free plan, Pay-per-Boost, Promoted badge, basic promoted sorting; subscription планови во база (online payment = future).

25. Future features

Android/iOS app · background tracking · online плаќање · wallet · refund policy · провизија · referral · premium drivers · verified ID/license · female-only rides · student/corporate rides · automatic route matching · AI pickup suggestions · loyalty · promo codes · emergency/SOS · voice call masking · Viber/WhatsApp notifications · airport rides.

26. Препорачан tech stack

Frontend : Next.js 14+ (App Router) · TypeScript · Tailwind CSS · shadcn/ui

State/data: React Query (TanStack) · Zod валидација · React Hook Form

Backend : Supabase (PostgreSQL + Auth + Realtime + Storage + Edge Functions)
(алтернатива: custom Node/NestJS + Postgres + Redis)

Realtime : Supabase Realtime (chat, tracking, notifications) / WebSockets

Maps : Mapbox GL JS + Directions + Geocoding (или Google Maps + Places)

Geo : PostGIS за route-matching и detour

Email : Resend / SMTP transactional

SMS OTP : Twilio / локален SMS gateway

Hosting : Vercel (frontend) + Supabase (managed) / VPS

Push : Web Push / FCM (future)

Analytics: PostHog / Plausible (опц.)

27. Folder structure (Next.js App Router)

macedonia-ride/

```
├── app/
│   ├── (public)/      page.tsx, find-ride, offer-ride, how-it-works,
│   │                  safety, popular-routes, login, register,
│   │                  forgot-password, terms, privacy, contact
│   └── (auth)/        verify-email, verify-phone, onboarding/role
```


— dashboard/	
— passenger/	find, trips, trips/[id], bookings, messages, saved-locations, reviews, profile
— driver/	create-trip, trips, trips/[id], requests, active-trip/[id], vehicles, subscription, promotions, messages, reviews, profile
— admin/	users, drivers, passengers, trips, bookings, reports, cities, routes, subscriptions, promotions, settings
— trip/share/[token]/	
— api/	auth/, trips/, bookings/, search/, geo/, messages/, reviews/, reports/, notifications/, plans/, subscriptions/, admin/
— components/	
— ui/	(shadcn)
— rides/	RideCard, FilterPanel, BookingRequestForm ...
— maps/	RouteMap, LocationPicker, LiveTrackingMap
— chat/	reviews/ driver/ admin/ layout/
— lib/	
— supabase/	client.ts, server.ts, middleware.ts
— geo/	detour.ts, matching.ts, directions.ts
— ranking/	search-ranking.ts
— auth/	guards.ts
— validations/	zod schemas
— notifications/	
— hooks/	useLiveTracking, useGeolocation, useRealtimeChat
— types/	database.types.ts (Supabase gen), domain types
— supabase/	
— migrations/	*.sql (schema, RLS, triggers, functions)
— seed.sql	cities + popular routes + plans
— functions/	edge functions (promotion-expiry, notify)
— middleware.ts	
— tailwind.config.ts	
— next.config.js	

Seed (cities & popular routes):

Градови: Скопје, Велес, Куманово, Тетово, Гостивар, Охрид, Битола, Прилеп, Штип, Кочани, Струмица, Кавадарци, Неготино, Гевгелија, Струга, Ресен, Кичево, Дебар, Радовиш, Свети Николе, Делчево, Берово.

Популарни рути (за featured/SEO): Скопје↔Велес, Скопје↔Битола, Скопје↔Охрид,

Скопје↔Куманово, Скопје↔Тетово, Скопје↔Штип, Скопје↔Прилеп,

Скопје↔Гевгелија,

Скопје↔Струмица, Битола↔Охрид, Велес↔Штип, Кавадарци↔Скопје.

28. Практичен development plan по фази

ФАЗА 0 — Setup (1 недела)

Repo, Next.js+TS+Tailwind+shadcn, Supabase проект, миграции (секција 15),

seed cities/routes/plans, Auth (email+phone), middleware/guards, дизајн систем.

ФАЗА 1 — Core домен (1-2 недели)

Профили, улоги, vehicles CRUD, cities. RLS политики. Basic dashboards skeleton.

ФАЗА 2 — Trips & Search (2 недели)

Create-trip wizard + waypoints + detour settings + Mapbox рута/polyline. Search со hard filters + RideCard + два секции (promoted/all) basic ranking. Trip details (privacy-aware).

ФАЗА 3 — Bookings & Chat (2 недели)

Request-to-join, detour calc (PostGIS + Directions), accept/reject, seat decrement тригер, статуси, chat (Realtime), notifications (in-app+email).

ФАЗА 4 — Live tracking (1-2 недели)

Driver Start/Arrived/PickedUp/Complete, watchPosition, Realtime broadcast, passenger LiveTrackingMap + ETA, Share Trip, auto-off.

ФАЗА 5 — Reviews, Reports, Admin (2 недели)

Двонасочни reviews, reports flow, admin модули (users/drivers/trips/bookings/reports/cities/routes), KPIs.

ФАЗА 6 — Monetization (1-2 недели)

Plans во база, Pay-per-Boost, Promoted badge, promotion lifecycle cron, promotion stats, subscription (manual/cash активација).

ФАЗА 7 — Hardening & Launch (1-2 недели)

Rate limiting, validation, тестови, мобилен QA, перформанси, SEO на popular-routes, аналитика, soft-launch на 2-3 рути (Скопје↔Велес прв).

29. UI/UX предлог

Принципи: mobile-first, една примарна акција по екран, минимум кликови до „Request to Join“. Чисти картички, видлива цена и слободни места, доверба (рејтинг, verified badge, слика).

Дизајн: светла тема + опц. dark; примарна боја (пр. тиркизно/сино за доверба), акцент за promoted (суптилен, не агресивен). Македонски јазик прв, опц. албански/англиски (i18n). Големи tap targets, bottom nav на мобилен, sticky „Request“/„Publish“ копчиња.

Клучни екрани: Search (горе филтри што се collapse-ираат, листа картички, мапа toggle), Trip details (рута на мапа врв, потоа возач/авто/правила, sticky СТА), Create-trip wizard (прогрес индикатор, мапа во чекор за рута), Active trip (голема мапа + статус лента + ETA), Admin (DataTable + детал drawer).

Trust & safety во UI: verified badge, делумно скриена табличка, „телефон видлив по одобрување“, Share Trip копче истакнато, лесен пристап до Report.

30. Кои функционалности да се направат први (приоритет)

P0 (без нив нема производ):

Auth+улоги · Vehicles · Create trip + рута/waypoints · Search + ranking basic · Trip details (privacy) · Request-to-join + pickup/dropoff + detour calc · Accept/reject + seat logic · Статуси на патување · Chat.

P1 (комплетен MVP):

Live tracking (foreground) · Notifications (in-app+email) · Reviews · Reports · Admin core · Saved locations · Share Trip.

P2 (раст/монетизација):

Pay-per-Boost + Promoted badge + promoted sorting · Subscription планови · Promotion stats · Featured routes · Popular-routes SEO.

P3 (future): native app, background tracking, online плаќање, wallet, referral, verified ID/license, AI pickup, corporate/student, SOS.

Прв реален milestone: пушти една рута end-to-end (Скопје → Велес) со P0+минимум P1 (tracking + notifications), собери реални возачи/патници, па скалирај на останатите рути.

Крај на спецификацијата.